

CoSc 3311

Fundamentals of Software Engineering

Dagmawi Lemma (PhD)

Assistant Professor, AAU

courses.dag@gmail.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

1

Software Architecture

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

206

Software Architecture

- What is architecture?
- Architectural Styles
 - Tiered vs layered architecture
 - Open vs Closed architecture
- Selecting the right architecture
- Further readings
 - Enterprise Architecture
 - SOA

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

208

What is Architecture?

- High level description of the software with respect showing
 - How the low-level design elements (modules/subsystems) are arranged
 - How the structured solution meets all of the technical and operational requirements, while optimizing common quality attributes such as performance, security, and manageability

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

209

What is Architecture?

- K. Philippe et. al, derived and refined a definition of architecture based on work in (Shaw and Garlan 1996). According to their definition is:
 - Software architecture is about describing how the software is organized
 - The description may include
 - the structural elements and their interfaces by which the system is composed;
 - behavior as specified in collaboration among those elements;
 - composition of these structural and behavioral elements into larger subsystems; and
 - an **architectural style** that guides this organization

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

210

What is Architecture?

- Architecture focuses on how the major elements and components within an application are used by, or interact with, other major elements and components within the application
 - At low-level design, selection of data structures and algorithms or the implementation details of individual components are the concerns
 - However, some of the low-level design concern could be dictated by the high-level decision – the architecture
- Architecture and Design concerns very often overlap
 - In some cases, decisions are clearly more architectural in nature
 - In other cases, the decisions are more about design, and how they help you to realize that architecture

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

211

Why Architecture?

- Architecture helps to provide solid foundation for the software
 - failing to consider key scenarios - *may affects functionality,*
 - failing to design for common problems - *may affects end-user criteria,*
 - failing to appreciate the long term consequences of key decisions – *may affect dependability and maintainability*
- Such failures put your application at risk

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

212

Why Architecture?

- Software should be designed with consideration for the
 - user,
 - system (the IT infrastructure), and
 - business goals
- For each of these areas, one should outline key scenarios and
 - identify important quality attributes (e.g., reliability or scalability) and
 - key areas of satisfaction and dissatisfaction (e.g., usability)

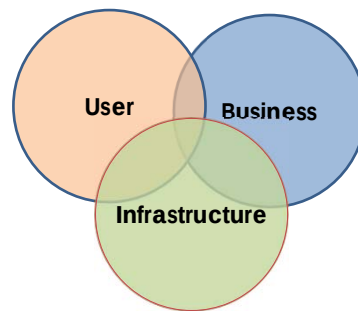
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

213

Why Architecture?

- It is needed to balance tradeoffs



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

214

Why Architecture?

- following high level concerns are suggested (in MSDN) to be considered when thinking about software architecture:
 - How will the users be using the application?
 - How will the application be deployed into production and managed?
 - What are the quality attribute requirements for the application, such as security, performance, concurrency, internationalization, and configuration?
 - How can the application be designed to be flexible and maintainable over time?
 - What are the architectural trends that might impact your application now or after it has been deployed?

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

215

Architectural Styles

- An architectural style, also known as architectural pattern, is a set of principles – a coarse grained pattern that provides an abstract framework for a family of systems
- Architectural styles describe
 - deployment patterns,
 - some describe structure and design issues, and
 - others describe communication factors
- Include patterns such as
 - client/server,
 - layered architecture,
 - component-based architecture,
 - message bus architecture, and
 - service-oriented architecture (SOA)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

216

Architectural Styles

- More specifically, an architectural style determines the vocabulary of components and connectors that can be used in instances of that style, together with a set of constraints on how they can be combined

David Garlan and Mary Shaw, January 1994, CMU-CS-94-166,
"An Introduction to Software Architecture"

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

217

Architectural Styles

- Architectural styles may vary by their key focus area
- Some of the major areas of focus and the respective architectural styles includes

Focus area	Architecture styles
<i>Communication</i>	Service-Oriented Architecture (SOA), Message Bus
<i>Deployment</i>	Client/Server, N-Tier, 3-Tier
<i>Structure</i>	Component-Based, Object-Oriented, Layered Architecture
<i>Domain</i>	Domain Driven Design

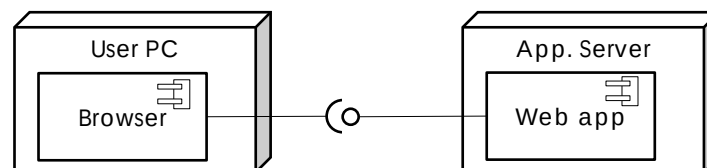
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

218

Tiered vs Layered

- Tier focus on the definition of the deployment style while layer refers to the structure of the software with respects to the function/service to be provided
- For example, in a web application, the web browser and the application server are deployed at different location – tiers; hence we have two tiers
 - In this example since we have a browser that request a service from the application server it can be referred as a client/server architecture – hence 2-tiered



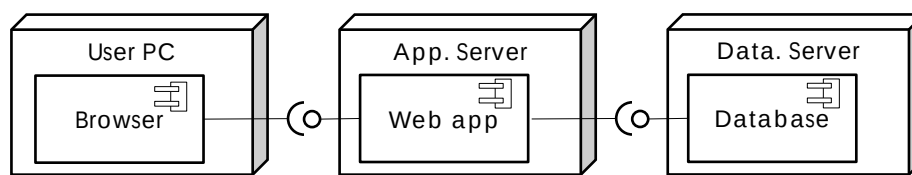
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

219

Tiered vs Layered

- It is also common the split servers based on their service (e.g. database, security, etc.); this could lead to another level of tier so we can have the third tier in such arrangement



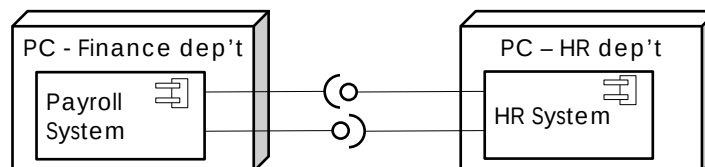
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

220

Tiered vs Layered

- Also, if we have applications deployed at different peers where each has their complete implementation but might support one another we can have peer-to-peer (p2p) architecture, so for n peers existing in the system we can have the p2p can form n-tier architecture



NB: being p2p does not justify n-tier, even in client/server configuration we can have n-tier as long as we have more than one server (deployment location)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

221

Tiered vs Layered

- Layer on the other hand focus on the structural configuration of the software, such as functionalities, regardless of the deployment location
 - Functionality within each layer is related by a common role or responsibility
- For example, continuing the web application example, perhaps the designer can decide every data request should be validated before the access to the data is permitted

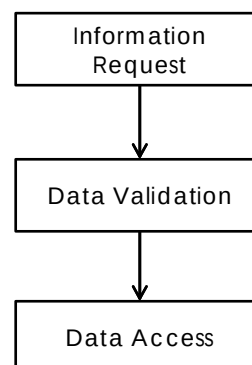
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

222

Tiered vs Layered

- In this arrangement, let us assume that after the data request is made the validation has to be done by the 2nd layer before it returns the requested data
- Where should it be validated would be the focus of tier (client – java script, server – server script (JSP, ASP)); on the other hand the functionality, when to validate and to whom to return the requested information is structural defined functionality, hence layer perspective



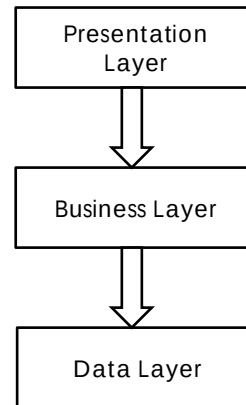
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

223

Tiered vs Layered

- The layers of an application may reside on the same physical computer (the same tier) or may be distributed over separate computers (n -tier)
- The components in each layer communicate with components in other layers through well-defined interfaces
- For example, a typical Web application design consists of
 - a presentation layer (functionality related to the UI),
 - a business layer (business rules processing),
 - and a data layer (functionality related to data access, often almost entirely implemented using high-level data access frameworks)
- Example

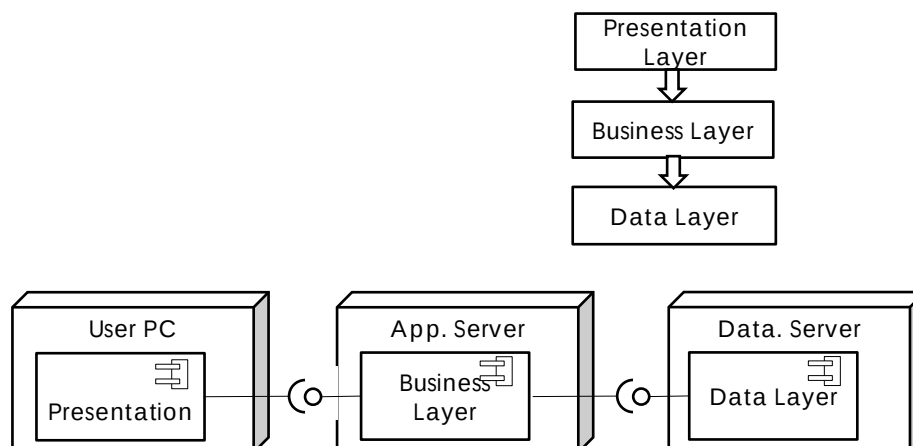


Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

224

Tiered vs Layered



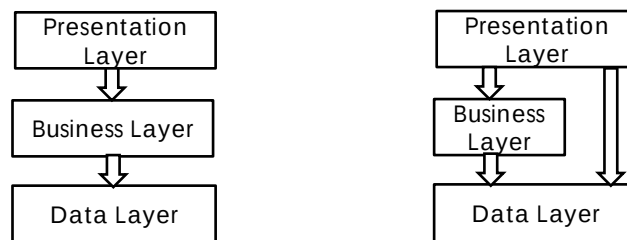
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

225

Open vs Closed

- **Closed Layered Architecture**
 - components in one layer can interact only with components in the same layer or with components from the layer directly below it
- **Open Layered Architecture**
 - allows components in a layer to interact with components in the same layer or with components in any lower layer



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

226

Selecting the Right Architecture

- As discussed in Chapter 1, software design requires technical skill; this includes knowledge on the existing types of architectural styles so that one can compare and contrast them prior to selection
- Selection of the architecture is also guided by the design goal
 - For example, the overall user experience of the solution is very often a function of the business and the IT infrastructure, and changes in one or the other can significantly affect the resulting user experience. Similarly, changes in the user experience requirements can have significant impact on the business and IT infrastructure requirements. Performance might be a major user and business goal, but the system administrator may not be able to invest in the hardware required to meet that goal 100 percent of the time. A balance point might be to meet the goal only 80 percent of the time

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

227

Client/Server

- A 2-tier architecture where one tier (client) request service and the other tier (server) provides the requested service
- Other variation of client/server may include
 - P2P architecture
 - Repository architecture
 - Application server
- The main benefits of the client/server architectural style are:
 - Higher security
 - Centralized data access
 - Ease of maintenance

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

228

Layered Architectural Style

- Focuses on the grouping of related functionality within an application into distinct layers that are stacked vertically on top of each other (as discussed in the previous slides)
- Common principles for designs that use the layered architectural style include:
 - **Abstraction:** Important to abstract the view of the system as whole while providing enough detail to understand the roles and responsibilities of individual layers and the relationship between them.
 - **Encapsulation:** No assumptions need to be made about data types, methods and properties, or implementation during design, as these features are not exposed at layer boundaries
 - **Clearly defined functional layers:** The separation between functionality in each layer is clear.
 - **High cohesion:** Well-defined responsibility boundaries for each layer, and ensuring that each layer contains functionality directly related to the tasks of that layer,
 - **Reusable:** Lower layers have no dependencies on higher layers
 - **Loose coupling:** Communication between layers is based on abstraction and events to provide loose coupling between layers

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

229

Layered Architectural Style

- The main benefits of the layered architectural style, and the use of a Separated Presentation pattern, are:
 - **Abstraction:** Layers allow changes to be made at the abstract level. You can increase or decrease the level of abstraction you use in each layer of the hierarchical stack
 - **Isolation:** Allows you to isolate technology upgrades to individual layers in order to reduce risk and minimize impact on the overall system
 - **Manageability:** Separation of core concerns helps to identify dependencies, and organizes the code into more manageable sections
 - **Performance:** Distributing the layers over multiple physical tiers can improve scalability, fault tolerance, and performance
 - **Reusability:** Roles promote reusability.
 - For example, in MVC, the Controller can often be reused with other compatible Views in order to provide a role specific or a user-customized view on to the same data and functionality.
 - **Testability:** Which is achieved as the result of decomposability

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

230

Component-Based

- Describes a software engineering approach to system design and development - CBSE
- Focuses on the decomposition of the design into individual functional or logical components that expose well-defined communication interfaces containing methods, events, and properties
- This provides a higher level of abstraction than object-oriented design principles, and does not focus on issues such as communication protocols and shared state
- Key Principles
 - Reusable
 - Replaceable
 - Not context specific
 - Extensible
 - Encapsulated
 - Independent

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

231

Component-Based

- Components depend upon a mechanism within the platform that provides an environment in which they can execute, often referred to as *component architecture*
 - Examples are
 - the component object model (COM) and the distributed component object model (DCOM) in Windows;
 - Common Object Request Broker Architecture (CORBA) and Enterprise JavaBeans (EJB) on other platforms
- Component architectures manage the mechanics of locating components and their interfaces, passing messages or commands between components, and – in some cases – maintaining state

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

232

Selecting Architectural Style

- Note that architecture is a high level description of the system showing how the system is organized.
- In the case of the system being software, the architecture aims to describe how the various elements of the software are organized.
- Note that software is a set containing elements where the elements can be:
 - User interfaces (UIs),
 - Processes (Ps), and
 - Business logic,
 - Environment operation,
 - Control, etc.
 - Various Data (Ds)
- in the case of our example, the Member Management Subsystem might contain many UIs, Ps and Ds and they have to be listed
- Once we identify the elements from our design, we have to consider the nonfunctional requirements while organizing the elements in layers or tiers and so on ...

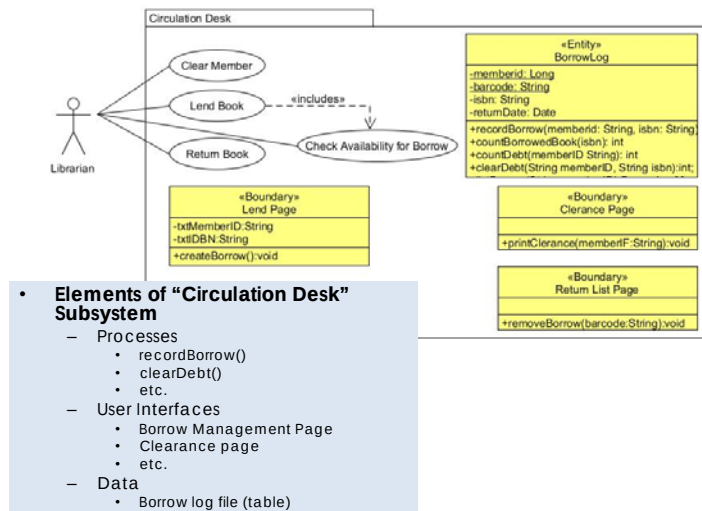
$$S: \{u_1, u_2, \dots, u_n, p_1, p_2, \dots, p_n, d_1, d_2, \dots, d_n\}$$

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

233

Example



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

234

MVC

- In Model/View/Controller (MVC) architecture, the system/subsystem elements are classified into three:
 - model - responsible for maintaining domain knowledge (business logic and data)
 - view - responsible for displaying the result of business logic and data to the user
 - Controller - responsible for managing the sequence of interactions with the user

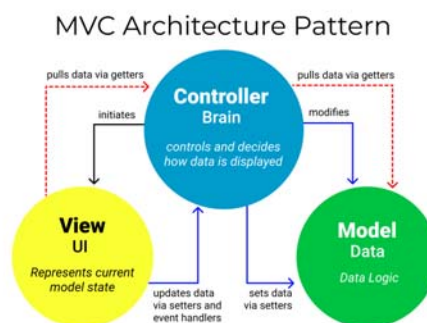


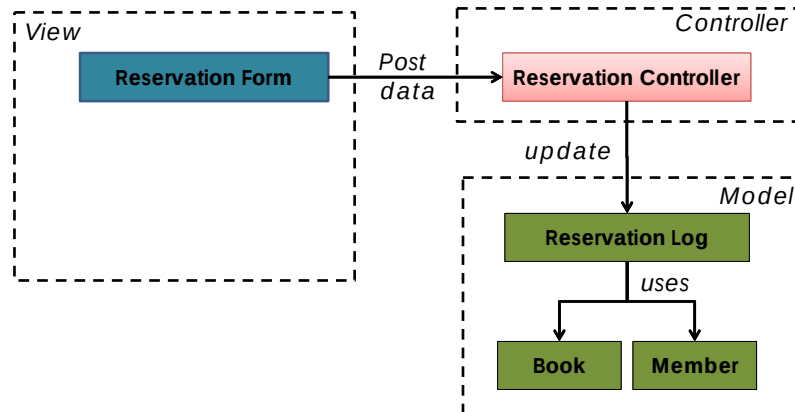
Image Source:
<https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

235

Example

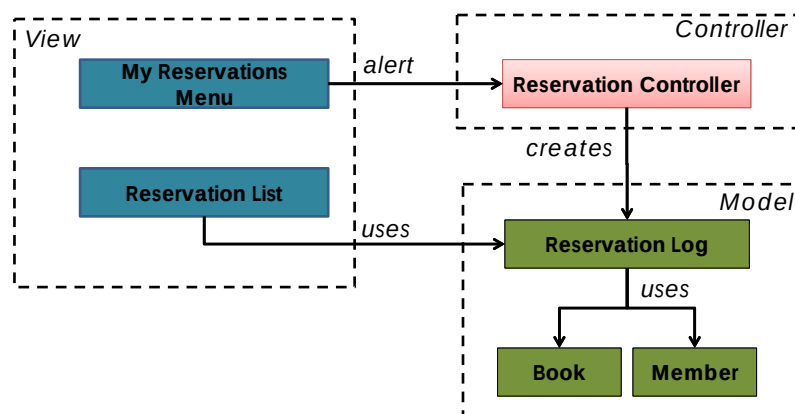


Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

236

Example

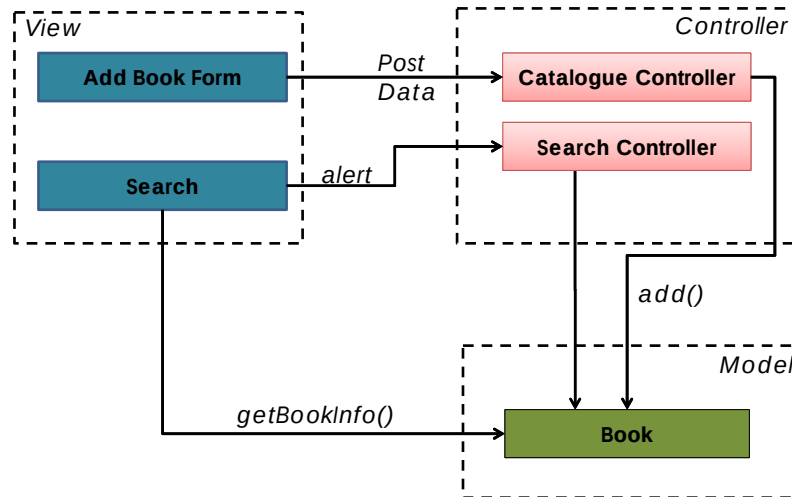


Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

237

Example



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

238